FRED TALKS

29th October 2026

CONTENTS

Please for the love of all that is holy: Stop writing long boring Titles

SWYX · 1431 WORDS

"You Said No MCP!"

EARENDIL · 1491 WORDS

What requirements don't you have?

SEANGOEDECKE.COM RSS FEED · 945 WORDS

Ownership

THORSTEN BALL · 1122 WORDS

Please for the love of all that is holy: Stop writing long boring Titles

SWYX · 30 MAR 2026 · SOURCE

People code switch. Every ethnic minority has lived experience of this. When you talk to me in Singapore, I sound very different than when I'm in San Francisco. And the way you talk at work is probably very different from how you talk to your kids and how you talk to your lover.

I've been dealing with a particularly virulent form of code switching in the simple form of titling blogposts and talks. When you ask the writer face to face what their thing is about, they can give you a perfectly human explanation. And then when they send you their piece, it's all "***Leveraging GenAI with Robust Data Foundations to Transform Businesses***". I've left my body by the time I read the first word.

My hope in writing this piece is to **put an end to bad titles**.

If I sent you this guide, it is not an insult. You just don't have good title game. That's ok. You're great at what you do, title game isn't your job. I'm the guy that wrote How To Name Things and Two Words. I sent you this guide ***because*** your shitty title is the main thing stopping your readers from getting to the good part. If you didn't have anything good, I wouldn't bother with a lost cause.

First…

Towards Leveraging Corporate Speak For Maximum Enterprise Synergy

DO NOT:

- Start with "Leveraging" or "Towards" or "Navigating" or "Empowering"

Yeah just stop. There's no rescuing this when you start there. Just delete meaningless corporate weasel words. Anything >3 syllables is a red flag.

Make It Quotable

If you do not try to make your title roll off the tongue, it probably won't.

Literally imagine telling your friend about your talk. Telling a new acquaintance to google your blogpost. Having a fan of yours cite it back to you. How much do they stumble? Yeah. This has all happened to me.

There's an extremely high correlation between what are considered the best talks and the quotability of titles. Simple Made Easy. Inventing on Principle. Our list of best demos/pitches of all time.

Resist colons. It lacks confidence. Why Use Lot Word: ~~When Few Word Do Trick~~? You're not High School Musical: The Musical: The Series.

Put the SEO keyword vomit in the description.

Titles are for humans.

Prescribe A Strong Opinion

Opinions are one of the most quotable things.

But more importantly, a good Opinion Piece is load bearing because it is eminently forwardable, and therefore viral and impactful. Your Job To Be Done is to be the definitive, conclusive, piece on that Opinion. Stake out territory in Opinion Space, list out all the arguments for, counter all the arguments against, provide datapoints, and watch that go further than anything you've ever done.

How to know what's a good opinion? **Three Strikes rule**: If you repeat yourself three times, ship it. Even if you're just quoting others, its fine, credit them and make it your own.

Mental Models and Frameworks are more complex opinions that allow others to scaffold their thought around you. My Radiating Circles and Measuring Developer Relations and Third Age of JavaScript did this.

Opinions contrary to an established opinion are the hardest to do well and the best when correct. What Clayton Christensen Got Wrong made Ben Thompson's career. Rise of AI Engineer countered the prevailing Prompt Engineer -and- GPU Rich Startup/ML Engineer narrative at the time, with room to grow.

Curiosity Gap Dynamics

Marketers know the power of the Curiosity Gap. Here is where we toy closest with clickbait. I'll be honest that I wasn't even sure if I wanted to give it its own subheading, but its a very important concept that you should at least be aware exists, even if you're going to ignore it.

Strong enough opinions create their own curiosity gaps; you won't have to try very hard to make it interesting if you got my attention in 4 words anyway.

Curiosity gaps done poorly can lead to "burying the lede", which make your talk less Quotable and you also don't want to do that.

Simplest way to put it is; if the most interesting thing about your content is your title, and what people take away after having read your post matches what they would have predicted after reading your title, then you've probably said too much and could cut.

Play Madlibs

You don't have to invent your title from scratch (nor should you always do what I'm about to suggest). Here's some templates that work well:

- {*BENEFIT EVERYONE WANTS*} **without** {*DOWNSIDE EVERYBODY EXPECTS*}
  - e.g. How to Get Rich (Without Getting Lucky), How to Market Yourself (Without Being A Celebrity). See also subtitle of this post.
- {*PRESCRIPTIVE OPINION*}, **don't** {*THING YOU DO*}
  - e.g. Parse, don't validate
  - alternative prescriptive opinion: **Never** {THING YOU DO}
  - see also title of this post
- {*POPULAR THING*} **IS DEAD**
  - unfortunately this is clickbait that works
  - e.g. Prompt Engineering is Dead, Big Data is Dead
  - when given at a conference for *POPULAR THING*, this is known as a Heresy Talk

    - opposite also works:
        - **The Rise of** {THING}
        - **The Unreasonable Effectiveness of** {THING}
- **Fundamentals of** {THING}
    - straightforward, people love fundamentals
    - also **How** {THING} **Really Works**
    - You can also flip it and go for the **Advanced** {THING} tier content
- {CONVEY EXPERIENCE/SCALE} with {THING}
    - e.g. What We Learned from a Year of Building with LLMs, Insights from over ***10,000*** comments on "Ask HN: Who Is Hiring" using GPT-4o

There's loads more templates that work, please suggest in comments and I can add/review.

You can also browse the masters of microcopy like Steph Smith and Shaan Puri, and if you are worried about clickbait, you can see Veritasium's take or Simon Willison's Highlights.

also Andrew Chen senpai:

Understand Buzzword Metagame

Buzzwords work. Because of a mix of human psychology and algorithmic recsys. Don't avoid them, just use them judiciously. This involves the ability to read the room, which many smart people sadly cannot without significant involvement in the broader community you are trying to speak to. I don't see any way around it other than trying to authentically immerse, or consult a friendly who does.

Not all buzzwords are created equal. Databricks/Berkeley tried to push Compound AI Systems, Gartner is pushing Composite AI and Composable business — this resonates with their audience, but not more broadly.

Pick the shibboleths that work for the community you want. AI Engineer worked for reasons that are well understood now. GraphRAG is working outside of the company that coined it. These buzzwords travel, go ahead and put them in there.

Buzzwords rise and fall. Try to catch them on the rise — go harder than everyone else on the rise up and regularly test their effectiveness, such that by the time your main buzzword has tapped out you've got 2-3 others cooking.

Workshopping your titles

I write this post out of frustration, but also to capture my own thought process. I'm currently procrastinating while writing a Latent Space post.

- The original title was Why every AI Engineer needs an AI Gateway. Good, genuine opinion, load bearing. But not super memorable.
- My next iteration was "**Stop writing model routers, use an LLM Gateway**". Ok, prescriptive, evocative. But didn't cover the other elements other than model routing. One could shorten to "Stop writing your own LLM Gateway" but that doesnt quite express what people do.
- My next one is "**It's not your job to DIY an LLM Gateway**". No buzzword bc "your job" is implied (for a Latent Space audience), a little curiosity gap, LLM buzzword, strong opinion. But still not strong enough.
- Final one is "**LLM Gateway: The One Decision That Removes 100 AI Engineering Decisions**". Leans on Tim Ferriss' madlbs.

It's very common for the "tail to wag the dog to wag the tail" - the title is so important to clickthrough and so impactful to the message, that you spend a bunch of time on just the title itself, because that determines the piece you will write, then write the piece, then you go back and tweak the title based on what happened.

Amateur writers think their content has 95% of the value and then spend 5% on the title. Pros will put the value of titles closer to 50% and then act like it.

You Should Break The Rules Once You Understand Human Behavior More Intuitively Than Rules Can Model

See title of this post. No better aura than intentional, conspicuous, successful rulebreaking.

Subscribe to our newsletter

“You Said No MCP!”

EARENDIL · 29 OCT 2026 · SOURCE

If you went to pi.dev in the past, you found a proud declaration that Pi does not support MCP. If you listen to podcasts where we talked about Pi, you will have found more than one dismissive statement about MCP from us. Including a post by Mario about it. And yet, if you upgrade to Pi you will find MCP is now a supported piece of functionality. What happened?

Things Change

The first thing to remember is that the world is not static. We have been paying attention to MCP over the last year and the MCP of today is not the MCP of yesteryear. That alone would not be much of a reason to put it into the core, however. As you know, Pi has a great ecosystem of extensions, surely MCP could have been an extension? Maybe even an Earendil endorsed extension. And yes you are indeed correct in that MCP could have been an extension, as it was. That MCP is now part of the core is a result of us putting our heads together and rethinking it.

What Exactly Changed?

The reason we brought MCP into the core is not just about how MCP has changed, but also because we found that the changes it would require were generally useful. For example, the changes we have made to MCP also enable the use of Jev more easily within Pi. Ultimately what Pi needs is quite similar to what MCP needs: a sandbox to play with in the form of an interpreter.

While a lot of things have improved about MCP, quite a few have not. The biggest issue with MCP continues to be that it’s hard to compose. Even with codemode, which is just a neat little sandbox to allow composing of tool calls, MCP doesn’t fully deliver on this. But that at this point is less the problem of MCP but the MCP servers out there and different approaches of harnesses to work with them.

Many MCP servers are still built for harnesses that just dump tools into the context and are trying to optimize on their side for token efficiency by returning text. The way we like to think about MCP at this point is that it should be much closer to OpenAPI with intelligent tool discovery. That means tools should return structured data and tools should be discoverable by their documentation and description.

The reason CLIs are so functional is that the agent and model just wire stuff together with efficient bashisms. But there is no fundamental reason why you can't do that with MCP either. MCP in Pi is just built on exposing those tools to a JavaScript sandbox like other harnesses like Codex do too.

MCP in a Modern LLM

This will raise the question why we didn't just do Codemode without MCP. Part of the answer to this has to do with how tools are expressed in Pi today. We did a lot of work in recent months to allow Pi to make sense with new models that allow deferred tool loading, mid-conversation system messages and reasoning level changes. However we did not yet upgrade our tool loadout to better scale to these new capabilities.

In a Codemode world one needs to decide if the tool is available to the LLM or only the codemode part of the LLM. A normal MCP extension does not have enough metadata available from Pi's tool loadout to make that experience work well. So we needed to ensure that tools can be configured to just be deferred or be a Codemode specific thing.

And while we could have just wired up the metadata to enable better MCP extensions, we also think that MCP with Codemode solves quite a few of the issues that it traditionally had. We believe the best way to positively influence something is to embrace it. And while we think that modern MCP is in a much better spot than MCP ever was, the servers and patterns still leave room for improvement. So we want to be part of that conversation and help shape it to work well in small harnesses instead of standing on the sidelines and just watching.

What Is Codemode?

Now we talked so much about Codemode, it might be worth explaining what that even is. When a harness executes tools, for the most part it has two sides: it can do it where bash runs, or it can do it where the harness agent loop runs. The trust level on both sides is very different. The harness loop quite often runs in an environment that is trusted, whereas the tools it executes often run within a sandbox that is not really all that trusted.

Codemode is special in that it runs where the harness runs. It's best understood as a mechanism to orchestrate and coordinate tool calls. It's a sandbox that allows an agent to issue those tool calls in a way that gives it more flexibility about in which order it should do it, and it allows it to use JavaScript to combine them together. Because Codemode also runs on the harness side, its state is also maintained as part of the session transcript instead of the file system.

Now in theory any language could do, but JavaScript is quite attractive as small versions of JavaScript can be shipped as WASM binaries and allow reasonable levels of protection.

In Pi, Codemode is automatically loaded when MCP is configured, or it can be added to the configuration as a default tool. Just ask pi to reconfigure itself to enable codemode! You can then use it for quite interesting things, not just MCP. For instance when you are logged in with a provider that provides "Jev" you can do things like this:

> Use typesafe/jev via codemode to find the 20 most frustrated commenters on our issue tracker

And it will cleverly combine things like the Linear MCP and Jev to do that analysis, right from within Pi, without wasting any context at all.

> Use typesafe/jev via codemode to find the most frustr

I'll pull the open issues from Linear and let Jev rate

```
codemode
const { issues } = await tools.mcp__linear__list_issues
  team: "Pi", state: "open", limit: 250,
});
const jev = await models.getModelOfType(
  "classifier", "cloudflare-workers-ai", "typesafe/jev"
);
const questions = {
  frustration: {
    type: "choice",
    instructions: "Judge ONLY the emotional tone of the
      "Ignore how severe the bug is.",
    criteria: {
      none: "Neutral, factual, or friendly, even about
      mild: "Explicit annoyance, impatience, or disappo
      high: "Clearly angry, exasperated, sarcastic, or
    },
  },
};

const results = [];
let next = 0;
async function worker() {
  while (next < issues.length) {
    const issue = issues[next++];
    const { comments } = await tools.mcp__linear__list_
      issueId: issue.identifier,
    });
    const c = await models.classify(jev, { state: { ...
    results.push({ id: issue.identifier, title: issue.t
  }
}
await Promise.all([worker(), worker(), worker(), worker
store("frustration", results);
```

```
const score = (r) => r.probabilities.mild * 0.5 + r.pro
const counts = {};
for (const r of results) counts[r.choice] = (counts[r.c
const flagged = results.filter((r) => r.choice !== "non
flagged.sort((a, b) => score(b) - score(a));
return {
  total: results.length,
  counts,
  flagged: flagged.map((r) => `${r.id} ${r.title}`),
};

... (331 earlier calls)
✓ mcp__linear__list_comments {"issueId":"PI-4714"} 255m
✓ mcp__linear__list_comments {"issueId":"PI-3200"} 354m
✓ models.classify cloudflare-workers-ai/typesafe/jev 75
✓ models.classify cloudflare-workers-ai/typesafe/jev 75

{
  "total": 167,
  "counts": {
    "none": 156,
    "mild": 11
  },
  "flagged": [
    "PI-7740 TUI after /reload does not follows custom
    "PI-9566 context size defaults to 128k despite the
    "PI-10031 Pi sporadically stuck in \"Working...\" w
    "PI-4714 [feature] /update TUI command instead of p
    "PI-5916 Support provider extensions with model ali
    "PI-7547 [Windows] [sink-thread] How do you use Pi
    "PI-6907 Add an Installation section to the README'
    "PI-6212 Proposal: Bedrock path should honor `compa
    "PI-9932 before_agent_start: forced system prompt k
    "PI-7730 High CPU usage on Mac OS with long session
    "PI-6600 pi update --extensions and npm scripts: np
  ]
}

Most of the tracker is calm. Jev rated 156 of the 167 c
```

```
as mildly frustrated and none as highly frustrated.

The clearest cases:

- PI-6907 no installation section in the README (“It's
Pi gets stuck on “Working...” after pressing Esc during
/update command request (“a pain in the butt”) - PI-773
in long sessions

The per-issue verdicts are stored in codemode under fru
into any of them without fetching the issues again.
```

A condensed replay of such a session in Pi.Skip

We will have more to say about things like Jev and Codemode later, but we hope this post serves as an example of how we continue to adapt and update Pi thoughtfully as the world continues to evolve.

What requirements don't you have?

SEANGOEDECKE.COM RSS FEED · 19 OCT 2026 · SOURCE

When designing software, the requirements you don’t have are as important as the requirements you do. Identifying what you can compromise on — what can be slow, or stale, or inflexible — is usually the key to an elegant design.

Elegant systems drop some requirements

[SVG OMITTED]

For instance, take preforking web servers like Unicorn. Because they lack the feature of shared mutable state between server instances, they can run each server in its own process, which provides isolation for free. Because Unicorn specifically ignores the problem of slow client handling, it can queue requests directly on Unix sockets, drastically simplifying the implementation.

Because S3 does not support atomic multi-key updates — you cannot update multiple blobs in a single transaction — it can store data at the blob level however it chooses. That makes it much faster, since reads and writes don't have to cross storage partitions, and much easier to scale, since less coordination means fewer sources of truth.

StatsD uses UDP by default, which means it doesn't guarantee that every piece of data you send makes it into storage. In exchange for dropping that requirement, it incurs much less network overhead, which is a big advantage for a stats aggregation service that's constantly shoveling data over the wire.

The general design approach of fanning out data to an edge cache buys a lot of speed and scaling by dropping the requirement that data must always be up-to-date. SQLite buys operational simplicity — it can run out of a single file! — by dropping the requirement of concurrent writes. By accepting high tail latency, Cursor can scale their distributed git backend more easily. Well-designed systems turn their weaknesses into strengths.

Why dropping requirements is hard

[SVG OMITTED]

Anyone who's spent time designing real-world software (or who's studied distributed systems) knows that system design is a process of compromise. If you attempt to satisfy every single potential requirement — speed, concurrency, up-to-date data, a simple interface, supporting complex user behaviors — you will fail to design a working system at all.

Figuring out what requirements you can relax is difficult because **it combines both business and technical expertise**. It is technically challenging to identify precisely the level of staleness that a particular design change will allow; it is likewise challenging from a business perspective to determine whether that level of staleness is acceptable to users.

In most cases, an engineer who understands the technical side must try and explain it to a product manager, who can make a judgement on the business side. But this is a very difficult partnership to get right. Engineers and PMs have different incentives and little ability to check each other's work, which makes it hard to build trust. Without trust, the product manager is liable to avoid giving any ground at all, which leaves the design in an impossible state.

This is an underappreciated reason why solo developers or startups can sometimes run rings around larger companies. Because the technical and business decisions are united in a single person, they can make design tradeoffs that would be impossible for a large company, enabling them to build systems that are far more elegant and powerful.

Working with LLMs

[SVG OMITTED]

Knowing what requirements you don't have becomes far more important when working with coding agents. Frontier models are very capable, but they have all gone through the purifying flames of RLVR: like high-achieving schoolchildren, they are obsessed with satisfying the Grader. On top of that, they have very little business context compared to an experienced human. That often manifests as them catching correctness or consistency "bugs" that are really just hypothetical unsatisfied requirements.

For instance, it's not uncommon for an LLM to casually introduce a mutex or other concurrency mechanism to ensure that two pieces of semi-related data are kept in sync, even if not told to do so. I've seen LLMs add weird checks in the middle of request handling to ensure that some already-checked piece of data hasn't changed mid-flight, even if that wouldn't successfully prevent the feared TOCTOU bug. LLMs are obsessed with edge cases like "what if the current user loses their admin status in the middle of us handling one of their admin requests?", or "what if the cache in storage partition A is updated a nanosecond before the one in storage partition B, leading some users to see different data?". Particularly if run in a loop or told to "make no mistakes", they will pile on cases like these until they've transformed your four-line change into a thousand-line monster.

This is why I'm confused by some people's claims that code review is dead. About half the time I review a large AI-generated change, I notice some unnecessary requirement that makes the diff more complex. When I dig deeper, the LLM invariably introduced this requirement on its own.

When working with a cache, I will often spell out in my prompt that I don't mind the data sometimes being stale or briefly inconsistent between cache instances. When reviewing Golang code, the first thing I do is scan for added mutexes or layers of indirection that don't look necessary to me.

If you allow requirements like these to propagate unchecked through a codebase, they will petrify the system over time. The whole thing will be such a mess of checking and re-checking and holding locks and enforcing consistency that it cannot be scaled at all. Keeping systems elegant requires a ruthless focus: on accepting stale data or occasional tail latency, on avoiding all but critical instances of coordination, and in general on figuring out what requirements you *don't* have to satisfy.

Ownership

THORSTEN BALL · 08 JUL 2026 · SOURCE

Thoughts on ownership I sent to the Amp team in internal note

The following is an internal Slack message I sent to my Amp teammates after I had a conversation with one of them about ownership. It's only lightly edited.

I shared it before, but not here, because I didn't think too much of it. Then today, someone said their CTO shared my post with them and I thought: well, now I have to put it in the newsletter, don't I? So here we are.

Below, after the Slack post, I added some thoughts on juniors on how I see this advice applying to them.



Just had a (great) conversation about ownership and engineering here and I realized that I often use the phrase "ownership" or allude to it, but haven't explained what "ownership" means to me in a while.

So, ownership.

If I ask you "can you own this?" or "can you take care of this?" or "are you on it?" — what I'm doing is I'm asking you to *own* it, to own the solution of a problem from end to end. From "we have a problem" to "we don't have to think about it again."

That means, when you say that you're owning something, the expectation is that you…

- Think about what the problem actually is. Maybe you already have a solution in mind, without having thought about what we're actually trying to solve here. Maybe you think "the problem is that we need to migrate from using X to using Y", but that's not a problem, that's a solution. The problem is likely something like "performance is bad", "it's not stable", "it fails for customer x". Maybe there's other possible solutions to that? Think about those. What are the tradeoffs? What's the best solution to go with considering these tradeoffs?

- Think about edge cases. What are they? Which ones are important? Which ones can we ignore?

- Think about failures. Network failures are a given, for example. How do we handle them? Retry? Well, how often? How long?

- Think about data flow. How much data is involved here? Does data need to be migrated? Cleaned up? How can I get my hands on data to properly test this? What invariants are in the data? What assumptions do I have about the shape of the data that I haven't confirmed yet?

- Think about how you'd test this. How can I know that what I built is correct or not? Are tests enough? Do I need to manually poke at things? Is the difference visible on a screenshot or in a video?

- How would we announce this? How do we communicate it? Can you picture it? How does it fit into the larger picture of our roadmap? Questions or concerns in that area — push back! ask!

- Do the work, with precision, with care, with a sense of urgency, with calmness. Do not half-ass things. Before you merge, ask yourself: am I proud of this? would I show this to John Carmack and say "here's what I built, under these constraints, with these tradeoffs?"

- Test it manually. Yes, there's automated tests. But in 99% of cases you can manually test or confirm that what you built works: you can run it yourself, you can ask an agent to run through test scenarios, you can poke at the data before and after, you can take screenshots, you can make a demo. Are you sure that what you did actually solves the problem?

- Make sure it lands in production and *works in production*. Is it deployed? Did the deploy fail? Do you need to activate a feature flag? Does the feature flag work? Can you use it in production? Can you confirm it's actually deployed?

- If you think your colleagues needs to know about this change, because it's new feature they should all test, or it's a new convention in codebase, or maybe it's a tricky thing everybody needs to be aware of, or something else: let them know! Do not underestimate peripheral vision: knowing that person X yesterday changed the behavior of how Z works might save person Y three hours of debugging today when a bug report related to Z comes in.

- Do customers need to know? Who reported the bug? Who's blocked? Let them know.

- Does the world need to know? Announce that it's out.

- Are there follow-ups? Do you need to check on what you shipped in the logs? A week later maybe?

Yes, that's a lot. And there's actually more, because I'm sure I forgot some stuff.

But that's how you build a product in a small team. We don't have PMs, we don't have a Q&A department. We're small, but we're *great*, we can do all of that.

And it's always okay to ask for help, it's okay to ask questions, it's okay to redo things and triple-check. What's not okay is to implicitly assume that someone else will do the things here that you haven't thought about.



"How does this apply juniors? You can't expect them to really do all of that?"

I've been asked these questions, or variations of them, after I shared the thoughts above and here's my answer.

I do *not* expect juniors to *do* all of these things right away. But I would expect them to read through the list and *aspire* to one day be able to do all of these things. Until then, they can and should ask for help.

In fact, I don't expect *anybody* to *always* do *all of these things* for *everything*. It's a mental checklist of things to consider — problem, edge cases, tradeoffs, deployment, customers, messaging, … — but for quite a few things there aren't edge cases to consider. Or big tradeoffs to weigh. Or deployment is a solved problem. And maybe someone else does the messaging for you.

And I imagine that most of these things you shouldn't even consider when you work in a company with, say, 5000 employees. I've never worked in a company that large, only startups, so I can't speak to how to successfully ship a software feature at Apple, end to end.

But when you work in a small company in which there's only a single department, when you want to build things you're proud of, when you work with me and you say own something, I expect you to keep these things in mind and run through them before you declare something as done.



You know what *you* should own? A subscription to this newsletter: